

hexens x 1inch"

Security Review Report for 1inch

January 2026



Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
 - Security Review Lead
 - Scope
 - Changelog
4. Severity Structure
 - Severity characteristics
 - Issue symbolic codes
5. Findings Summary
6. Weaknesses
 - Token pair not bound when direction-only validation is used
 - Bidirectional PeggedSwap normalization mismatch in asymmetric configurations
 - Direction not enforced for 1->0-only adjustment instructions
 - Fee-on-Transfer Tokens Cause Accounting Mismatch and Maker Loss
 - Decay offset can cause an implicit revert on balanceOut underflow
 - Token Invalidators Assume Fixed Balance Semantics
 - The attempt to charge protocol fee may fail unreasonably
 - Static Balance Accounting Breaks Rebase Token Support in Aqua
 - Extruction Instruction Accepts Arbitrary Target Addresses Without Validation

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This report covers the security review for 1inch. This audit consisted of the new Aqua protocol and Swap-VM protocol, as well as updates to the Solidity-Utills repository.

Our security assessment was a full review of the code, spanning a total of 1.5 weeks.

During our review, we did not identify any major security vulnerabilities.

We did identify several minor risks and optimizations.

All reported issues were fixed or acknowledged by the development team and subsequently verified by us.

We can confidently say that the overall security and code quality have increased after completion of our audit.

3. Security Review Details

- Review Led by

Jahyun Koo, Lead Security Researcher

- Scope

The analyzed resources are located on:

-  ▪ Aqua: <https://github.com/1inch/aqua/tree/89cedfa608aa9fdc60d8cd936b76a540e4b9346f>
-  ▪ SwapVM: <https://github.com/1inch/swap-vm/tree/f014b3e71e88ed242900c8ad359ea1981848cbfe>
-  ▪ SolidityUtils: <https://github.com/1inch/solidity-utils/tree/38e5f8de705a8821237520ae06f3e9b0d40daf01>

The issues described in this report were fixed in the following commits:

-  ▪ Aqua: <https://github.com/1inch/aqua/tree/81c26e4619ce21556ab02b3284ee2685de21fb18>
-  ▪ SwapVM: <https://github.com/1inch/swap-vm/tree/b2daef83656b1ccbfc1c83d3df869c0ebb60badc>
-  ▪ SolidityUtils: <https://github.com/1inch/solidity-utils/tree/29043f22422fde454951e9733129cce5d67e6a39>

- Changelog

	12 January 2026	Audit start
	22 January 2026	Initial report
	6 March 2026	Revision received
	9 March 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

1. Impact of the vulnerability
2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

▪ Severity Characteristics

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical

Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.

High

Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium

Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.

Low

Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.

Informational

Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

▪ Issue Symbolic Codes

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

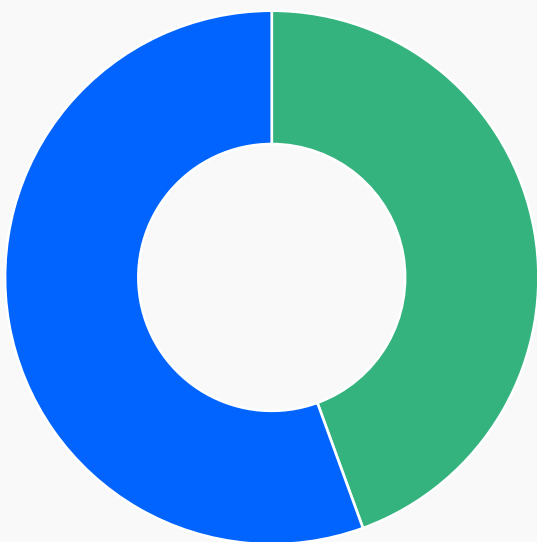
Severity

Number of findings

Critical	0
High	0
Medium	0
Low	4
Informational	5

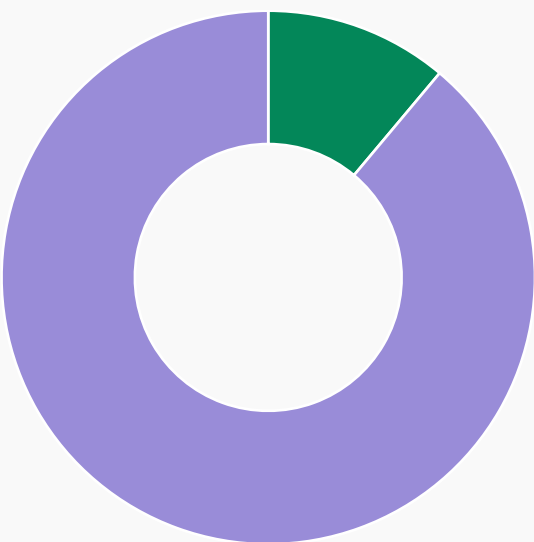
Total:

9



Low

Informational



Fixed

Acknowledged

6. Weaknesses

This section contains the list of discovered weaknesses.

OIN15-7 | Token pair not bound when direction-only validation is used

Acknowledged

Severity:

Low

Probability:

Rare

Impact:

Medium

Path:

```
swap-vm/src/SwapVM.sol
swap-vm/src/instructions/LimitSwap.sol
swap-vm/src/instructions/Balances.sol
```

Description:

The order hash does not include tokenIn or tokenOut, so the maker's signature does not bind a specific token pair. The LimitSwap instruction only validates address ordering (tokenIn < tokenOut) rather than the actual token addresses. This allows a taker to substitute a different token pair that satisfies the same ordering constraint.

The issue manifests when a Balances instruction includes more than two tokens. Consider a maker who wants to provide liquidity for multiple pairs through a single order. The maker configures a Balances instruction with three tokens(USDC, DAI, and WETH) and sets up LimitSwap with direction true (expecting tokenIn < tokenOut). The maker intends to trade USDC for WETH, but since all three tokens are in the balance list and DAI < WETH also satisfies the direction check, a taker can execute the swap as DAI → WETH instead. The maker's DAI balance is consumed rather than USDC.

This becomes more problematic when combined with external token approvals. If the maker has approved the contract to spend multiple tokens for different orders or purposes, a taker could exploit this by selecting a token pair that consumes allowance on an unintended token while still passing the direction validation.

```

bytes32 public constant ORDER_TYPEHASH = keccak256(
    "Order("
        "address maker,"
        "uint256 traits,"
        "bytes data"
    ")";
);

```

```

function _limitSwap1D(Context memory ctx, bytes calldata args) internal pure {
    require(ctx.swap.balanceIn > 0 && ctx.swap.balanceOut > 0,
        LimitSwapRequiresBothBalancesNonZero(ctx.swap.balanceIn, ctx.swap.balanceOut));

    bool makerDirectionLt = LimitSwapArgsBuilder.parse(args);
    bool takerDirectionLt = ctx.query.tokenIn < ctx.query.tokenOut;
    require(makerDirectionLt == takerDirectionLt, LimitSwapDirectionMismatch());

    if (ctx.query.isExactIn) {
        require(ctx.swap.amountOut == 0, LimitSwapRecomputeDetected());
        ctx.swap.amountOut = ctx.swap.amountIn * ctx.swap.balanceOut / ctx.swap.balanceIn; // Floor division
        for tokenOut is desired behavior
    } else {
        require(ctx.swap.amountIn == 0, LimitSwapRecomputeDetected());
        ctx.swap.amountIn = (ctx.swap.amountOut * ctx.swap.balanceIn).ceilDiv(ctx.swap.balanceOut); //
        Ceiling division for tokenIn is desired behavior
    }
}

```

```

function _staticBalancesXD(Context memory ctx, bytes calldata args) internal pure {
    require(ctx.swap.balanceIn == 0 && ctx.swap.balanceOut == 0,
        SetBalancesExpectZeroBalances(ctx.swap.balanceIn, ctx.swap.balanceOut));

    (uint256 tokensCount, bytes calldata tokens, bytes calldata initialBalances) =
        BalancesArgsBuilder.parse(args);
    bool foundTokenIn = false;
    bool foundTokenOut = false;
    for (uint256 i = 0; i < tokensCount; i++) {
        address token = address(bytes20(tokens.slice(i * 20)));
        uint256 initialBalance = uint256(bytes32(initialBalances.slice(i * 32)));
        if (token == ctx.query.tokenIn) {
            ctx.swap.balanceIn = initialBalance;
            foundTokenIn = true;
        } else if (token == ctx.query.tokenOut) {
            ctx.swap.balanceOut = initialBalance;

```

```
        foundTokenOut = true;
    }
}

require(foundTokenIn && foundTokenOut,
StaticBalancesRequiresSettingBothBalances(ctx.query.tokenIn, ctx.query.tokenOut, tokens));
}
```

Remediation:

Enforce token-pair binding in the program flow by validating tokenIn and tokenOut against an expected pair or allowlist before any swap amounts are computed. Alternatively, include the token pair in the order hash so execution parameters must match what the maker authorized.

Commentary from the client:

“The documentation reinforces mandatory token-binding patterns via balance instructions and clearly marks unsafe construction patterns.

<https://github.com/1inch/swap-vm/blob/9b9d821eb2f4ed8e2ffa00a4ccb161ca3cef7558/README.md#L518-L537>”

OIN15-6 | Bidirectional PeggedSwap normalization mismatch in asymmetric configurations

Fixed 

Severity:

Low

Probability:

Rare

Impact:

Medium

Path:

swap-vm/src/instructions/PeggedSwap.sol

Description:

In PeggedSwap.sol, `rateLt/rateGt` are selected at runtime based on token direction, but `config.x0` and `config.y0` are always used as provided. If a single config is used bidirectionally and `x0 != y0` (or `rateLt != rateGt`), the reverse direction normalizes reserves against the wrong baseline, which can skew the calculated output amount.

```
library PeggedSwapArgsBuilder {
    ...
    function getRates(
        Args calldata args,
        address tokenIn,
        address tokenOut
    ) internal pure returns (uint256 rateIn, uint256 rateOut) {
        if (tokenIn < tokenOut) {
            rateIn = args.rateLt;
            rateOut = args.rateGt;
        } else {
            rateIn = args.rateGt;
            rateOut = args.rateLt;
        }
    }
}

contract PeggedSwap {
    ...
    function _peggedSwapGrowPriceRange2D(Context memory ctx, bytes calldata args) internal pure {
        ...
        (uint256 rateIn, uint256 rateOut) = PeggedSwapArgsBuilder.getRates(
            config,
            ctx.query.tokenIn,
            ctx.query.tokenOut
        );
    }
}
```



```
);

...
uint256 targetInvariant = PeggedSwapMath.invariantFromReserves(
    x0,
    y0,
    config.x0,
    config.y0,
    config.linearWidth
);
```

Remediation:

Apply the same direction-aware mapping to **x0/y0** as is done for rates (swap them when the direction is reversed), or enforce a single allowed direction within the instruction so a config cannot be used for the reverse path.

OIN15-2 | Direction not enforced for 1->0-only adjustment instructions

Acknowledged

Severity:

Low

Probability:

Unlikely

Impact:

Low

Path:

swap-vm/src/instructions/OraclePriceAdjuster.sol#L86-L145

Description:

`OraclePriceAdjuster.sol`, `BaseFeeAdjuster.sol`, `DutchAuction.sol`, and `TWAPSwap.sol` are documented as 1->0 (token1 to token0) only, but they do not validate token direction at runtime. As a result, their internal calculations are applied based solely on **tokenIn/tokenOut** provided at call time. If a maker program does not enforce direction (for example, omitting a direction-gating instruction), a taker can swap in the opposite direction and still receive oracle, fee, auction, or TWAP adjustments. This can shift pricing away from the maker's intended configuration; for instance, `OraclePriceAdjuster` and `BaseFeeAdjuster` interpret prices as **token0** per **token1**, while `TWAPSwap` treats **balanceOut** and **minTradeAmountOut** as **token0** amounts.

```
function _oraclePriceAdjuster1D(Context memory ctx, bytes calldata args) internal view {
    require(ctx.swap.amountIn > 0 && ctx.swap.amountOut > 0,
OraclePriceAdjusterShouldBeAppliedAfterSwap());

    (
        uint64 maxPriceDecay,
        uint16 maxStaleness,
        uint8 oracleDecimals,
        address oracleAddress
    ) = OraclePriceAdjusterArgsBuilder.parse(args);

    // Get oracle price from Chainlink
    IPriceOracle oracle = IPriceOracle(oracleAddress);

    // Get latest price data from Chainlink
    (, int256 answer, , uint256 updatedAt, ) = oracle.latestRoundData();

    // Check if oracle data is fresh using configured staleness threshold
    // If maxStaleness is 0, skip the staleness check
    require(maxStaleness == 0 || block.timestamp <= updatedAt + maxStaleness,
OraclePriceAdjusterOraclePriceState(block.timestamp, updatedAt, maxStaleness));
```

```

// If oracleDecimals is 0, fetch from oracle (backward compatibility)
if (oracleDecimals == 0) {
    oracleDecimals = oracle.decimals();
}

// Convert oracle price to 1e18 scale using provided decimals
uint256 oraclePrice = answer.toUint256();
if (oracleDecimals < 18) {
    oraclePrice = oraclePrice * 10**(18 - oracleDecimals);
} else if (oracleDecimals > 18) {
    oraclePrice = oraclePrice / 10**(oracleDecimals - 18);
}

// Calculate current swap price (token0 per token1)
// Price = amountOut (token0) / amountIn (token1)
uint256 currentPrice = (ctx.swap.amountOut * 1e18) / ctx.swap.amountIn;

// Only adjust if oracle price is better for taker
if (oraclePrice > currentPrice) {
    // Oracle shows token0 is worth more token1, so taker should get better deal

    if (ctx.query.isExactIn) {
        // exactIn: Taker provides fixed token1, should get more token0
        // Increase amountOut proportionally, but cap at maxIncrease
        uint256 priceRatio = (oraclePrice * 1e18) / currentPrice;
        uint256 maxIncrease = (2e18 - maxPriceDecay); // Mirror of decay for increase
        uint256 adjustment = Math.min(priceRatio, maxIncrease);
        ctx.swap.amountOut = (ctx.swap.amountOut * adjustment) / 1e18;
    } else {
        // exactOut: Taker wants fixed token0, should pay less token1
        // Reduce amountIn proportionally, but cap at maxPriceDecay
        uint256 priceRatio = (currentPrice * 1e18) / oraclePrice;
        uint256 adjustment = Math.max(priceRatio, maxPriceDecay);
        ctx.swap.amountIn = (ctx.swap.amountIn * adjustment).ceilDiv(1e18);
    }
}

// If oracle price <= current price, no adjustment (already favorable for taker)
}

```

Remediation:

Add explicit direction validation to these instructions so they revert when called in the unsupported direction. This can be done by encoding an expected direction in their arguments or by enforcing a consistent token ordering check against **tokenIn/tokenOut**. Also ensure the instruction sequence guarantees direction gating before these adjusters execute, so the adjustment path is not reachable without a validated direction.

Commentary from the client:

“Direction requirements are documented as part of 1D strategy design and are enforced in LimitSwap.

<https://github.com/1inch/swap-vm/blob/9b9d821eb2f4ed8e2ffa00a4ccb161ca3cef7558/README.md?plain=1#L274-L280>”

OIN15-4 | Fee-on-Transfer Tokens Cause Accounting Mismatch and Maker Loss

Acknowledged

Severity:

Low

Probability:

Rare

Impact:

Medium

Path:

SwapVM.sol#L253-L259
 solidity-utils/contracts/libraries/SafeERC20.sol#L99-L126
 Invalidators.sol#L78-L84

Description:

SwapVM relies on `safeTransferFrom()` and assumes that the transferred amount equals the amount received. However, `safeTransferFrom()` only checks the success of the call - not the recipient's balance change. As a result, when interacting with FOT tokens, the protocol records and settles trades using the nominal amount rather than the actual received amount.

```

function _transferOrPull(
    address from,
    address to,
    address token,
    uint256 amount,
    bytes32 orderHash,
    bool useAqua
) private {
    if (useAqua) {
        AQUA.pull(from, orderHash, token, amount, to);
    } else {
        IERC20(token).safeTransferFrom(from, to, amount);
        // @audit: actual received amount is not verified
    }
}

```

```

function safeTransferFrom(
    IERC20 token,
    address from,
    address to,
    uint256 amount
) internal {

```

```
// ...
if (!success) revert SafeTransferFromFailed();
// @audit: only return value is checked, balance delta is not verified
}
```

Invalid Accounting in Invalidators

```
function _invalidateTokenIn1D(Context memory ctx, bytes calldata /* args */) internal {
    if (!ctx.vm.isStaticContext) {
        tokenInInvalidators
            [ctx.query.maker]
            [ctx.query.orderHash]
            [ctx.query.tokenIn] += ctx.swap.amountIn;
        // @audit: records nominal amount, not actual received amount
    }
}
```

Attack Path

1. A maker creates a limit swap order: 100 FOT → 10,000 USDC.
2. The FOT token charges a 2% transfer fee.
3. A taker calls **swap(order, FOT, USDC, 100, takerData)**.
4. SwapVM computes:
 - amountIn = 100 FOT
 - amountOut = 10,000 USDC
5. **_transferOrPull()** executes **safeTransferFrom(taker, maker, 100 FOT)**.
6. Due to the FOT fee, the maker receives only 98 FOT.
7. SwapVM does not detect the shortfall and proceeds to transfer 10,000 USDC to the taker.
8. The maker settles the trade based on incorrect accounting.

Remediation:

Consider using balance-delta checks to account for the actual received amount.

Commentary from the client:

“Fee-on-Transfer tokens are explicitly documented as unsupported due to accounting mismatch risk.

<https://github.com/1inch/swap-vm/blob/9b9d821eb2f4ed8e2ffa00a4ccb161ca3cef7558/README.md#L1589-L1593>”

OIN15-3 | Decay offset can cause an implicit revert on balanceOut underflow

Acknowledged

Severity:

Informational

Probability:

Rare

Impact:

Informational

Path:

swap-vm/src/instructions/Decay.sol#L81-L97

Description:

The `_decayXD` function subtracts the decayed offset from **balanceOut**. If the decayed offset exceeds **balanceOut**, the subtraction underflows and the call reverts with a generic panic. This behavior enforces an implicit invariant but is not made explicit, which can make failures harder to diagnose and reason about.

```
function _decayXD(Context memory ctx, bytes calldata args) internal {
    require(ctx.swap.amountIn == 0 || ctx.swap.amountOut == 0,
        DecayShouldBeCalledBeforeSwapAmountsComputation(ctx.swap.amountIn, ctx.swap.amountOut));

    // Adjust balances by decayed offsets
    uint256 period = DecayArgsBuilder.parse(args);
    ctx.swap.balanceIn += _offsets[ctx.query.orderHash][ctx.query.tokenIn][true].getOffset(period);
    ctx.swap.balanceOut -= _offsets[ctx.query.orderHash][ctx.query.tokenOut][false].getOffset(period);

    (uint256 swapAmountIn, uint256 swapAmountOut) = ctx.runLoop();

    if (!ctx.vm.isStaticContext) {
        _offsets[ctx.query.orderHash][ctx.query.tokenIn][false].addOffset(swapAmountIn, period);
        _offsets[ctx.query.orderHash][ctx.query.tokenOut][true].addOffset(swapAmountOut, period);
    }
}
```

Remediation:

Add an explicit guard that checks the decayed offset against **balanceOut** and reverts with a clear error, or adjust the logic to clamp the result if the intended behavior is to allow swaps to proceed with reduced liquidity.

Commentary from the client:

“Decay behavior is documented with explicit quote/swap divergence semantics and execution-order caveats.

<https://github.com/1inch/swap-vm/blob/9b9d821eb2f4ed8e2ffa00a4ccb161ca3cef7558/README.md?plain=1#L336-L341>”

Semantics

Severity:

Informational

Probability:

Unlikely

Impact:

Informational

Path:

`swap-vm/src/instructions/Invalidators.sol`

Description:

The `_invalidateTokenIn1D` and `_invalidateTokenOut1D` functions enforce cumulative fill limits by comparing accumulated amounts against `ctx.swap.balanceIn` and `ctx.swap.balanceOut` respectively. This design assumes these balance values remain constant throughout the order's lifecycle, which holds true when used with `_staticBalancesXD`.

However, when combined with `_dynamicBalancesXD`, this assumption breaks down. The dynamic balance mechanism updates stored balances after each swap: `balanceIn` increases by `swapAmountIn` while `balanceOut` decreases by `swapAmountOut`. As a result, the invalidation checks operate against a moving target rather than a fixed cap.

For `_invalidateTokenIn1D`, the increasing `balanceIn` causes the fill limit to become progressively more permissive with each swap. Conversely, for `_invalidateTokenOut1D`, the decreasing `balanceOut` makes the limit increasingly restrictive, potentially causing legitimate fills to revert unexpectedly.

This represents a configuration footgun rather than a direct vulnerability. Makers who unknowingly combine dynamic balances with token-based invalidators will experience fill limits that differ from their intended order parameters.

```
function _invalidateTokenIn1D(Context memory ctx, bytes calldata /* args */) internal {
    if (ctx.swap.amountIn == 0) {
        ctx.runLoop();
    }

    require(ctx.swap.amountIn > 0, InvalidateTokenInExpectsAmountInToBeComputed());
    uint256 prefilled = tokenInInvalidators[ctx.query.maker][ctx.query.orderHash][ctx.query.tokenIn];
    uint256 newFilled = prefilled + ctx.swap.amountIn;
    require(newFilled <= ctx.swap.balanceIn, InvalidatorsTokenInExceeded(prefilled, ctx.swap.amountIn,
        ctx.swap.balanceIn));
    if (!ctx.vm.isStaticContext) {
        tokenInInvalidators[ctx.query.maker][ctx.query.orderHash][ctx.query.tokenIn] = newFilled;
    }
}
```

```

}

function _invalidateTokenOut1D(Context memory ctx, bytes calldata /* args */) internal {
    if (ctx.swap.amountOut == 0) {
        ctx.runLoop();
    }

    require(ctx.swap.amountOut > 0, InvalidateTokenOutExpectsAmountOutToBeComputed());
    uint256 prefilled = tokenOutInvalidators[ctx.query.maker][ctx.query.orderHash][ctx.query.tokenOut];
    uint256 newFilled = prefilled + ctx.swap.amountOut;
    require(newFilled <= ctx.swap.balanceOut, InvalidatorTokenOutExceeded(prefilled, ctx.swap.amountOut,
    ctx.swap.balanceOut));
    if (!ctx.vm.isStaticContext) {
        tokenOutInvalidators[ctx.query.maker][ctx.query.orderHash][ctx.query.tokenOut] = newFilled;
    }
}

```

Remediation:

Ensure token-based invalidators are only used in conjunction with static balance flows where balance values represent fixed caps. Alternatively, modify the invalidation logic to reference a dedicated fixed limit field rather than the potentially mutable balance values.

Commentary from the client:

“Invalidator behavior and quote/swap divergence boundaries are documented as operational constraints for strategy authors and integrators.

<https://github.com/1inch/swap-vm/blob/9b9d821eb2f4ed8e2ffa00a4ccb161ca3cef7558/README.md#L336-L341>”

OIN15-1 | The attempt to charge protocol fee may fail unreasonably

Acknowledged

Severity:

Informational

Probability:

Likely

Impact:

Informational

Path:

swap-vm/src/instructions/Fee.sol#L79-L87

Description:

The `_protocolFeeAmountInXD` instruction charges a protocol fee for swaps by calling the `_feeAmountIn()` function. Subsequently, `ctx.swap.amountIn` is recalculated to include both the amount used for the swap and the protocol fee. Since the fee is included in `ctx.swap.amountIn`, it is effectively charged to the taker. This protocol fee is then transferred from the maker to the fee recipient.

```
function _protocolFeeAmountInXD(Context memory ctx, bytes calldata args) internal {
    (uint256 feeBps, address to) = FeeArgsBuilder.parseProtocolFee(args);
    uint256 feeAmountIn = _feeAmountIn(ctx, feeBps);

    if (!ctx.vm.isStaticContext) {
        IERC20(ctx.query.tokenIn).safeTransferFrom(ctx.query.maker, to, feeAmountIn);
    }
}
```

This instruction is executed before tokens are transferred between the taker and the maker. Because in the SwapVM, `ctx.runLoop()` is executed first to determine the final `amountIn` and `amountOut` results. After that, `_transferIn()` and `_transferOut()` are triggered to move the tokens based on those calculated amounts.

Therefore, if the maker's address does not hold `tokenIn` before the swap, the attempt to charge the protocol fee may fail because the `tokenIn` from the taker has not yet been transferred to the maker. This is an unreasonable failure for swap execution, since the maker should be the recipient of `tokenIn`, but is instead currently required to hold a balance of `tokenIn` beforehand for the swap to succeed.

Similar issues arise in the `_aquaProtocolFeeAmountInXD`, `_dynamicProtocolFeeAmountInXD`, and `_aquaDynamicProtocolFeeAmountInXD` instructions, as they all attempt to collect the protocol fee from the maker.

Remediation:

Consider collecting the protocol fee from the taker and excluding that fee from `ctx.swap.amountIn`.

Commentary from the client:

"The project documents that protocol-fee transfer depends on maker balance/allowance at execution time, and that quote/swap divergence can occur under these conditions.

<https://github.com/1inch/swap-vm/blob/9b9d821eb2f4ed8e2ffa00a4ccb161ca3cef7558/README.md#L343-L346>"

OIN15-5 | Static Balance Accounting Breaks Rebase Token Support in Aqua

Acknowledged

Severity:

Informational

Probability:

Likely

Impact:

Informational

Path:

aqua/src/Aqua.sol

Description:

Aqua treats balances as immutable after ship() and relies on its internal records rather than querying the token's actual balance. This design is incompatible with rebasing tokens whose balances change over time without transfers.

```
// Aqua.sol:40-52
function ship(
    address app,
    bytes calldata strategy,
    address[] calldata tokens,
    uint256[] calldata amounts
) external returns (bytes32 strategyHash) {
    strategyHash = keccak256(strategy);
    uint8 tokensCount = tokens.length.toUint8();

    for (uint256 i = 0; i < tokens.length; i++) {
        Balance storage balance =
            _balances[msg.sender][app][strategyHash][tokens[i]];
        balance.store(amounts[i].toUint248(), tokensCount);
        // @audit: stored as a static value; rebases are not reflected
    }
}
```

```
// Aqua.sol:30-38
function safeBalances(
    address maker,
    address app,
    bytes32 strategyHash,
    address token0,
    address token1
```

```

) external view returns (uint256 balance0, uint256 balance1) {
    (uint248 amount0, ) =
        _balances[maker][app][strategyHash][token0].load();
    balance0 = amount0; // @audit: returns stored value, not balanceOf()

    (uint248 amount1, ) =
        _balances[maker][app][strategyHash][token1].load();
    balance1 = amount1;
}

```

```

// Aqua.sol:63-70
function pull(
    address maker,
    bytes32 strategyHash,
    address token,
    uint256 amount,
    address to
) external {
    Balance storage balance =
        _balances[maker][msg.sender][strategyHash][token];
    (uint248 prevBalance, uint8 tokensCount) = balance.load();
    balance.store(prevBalance - amount.toUint248(), tokensCount);
    // @audit: internal record reduced without checking actual balance

    IERC20(token).safeTransferFrom(maker, to, amount);
    // @audit: reverts if negative rebase reduced maker balance
}

```

Attack Path

Scenario 1: Positive Rebase - Permanent Token Lock

1. Maker ships 1,000 stETH into Aqua.
2. Aqua records the balance as 1,000 stETH.
3. Over time, stETH positively rebases (e.g., +0.3%).
4. Maker's real wallet balance becomes 1,003 stETH.
5. Aqua still recognizes only 1,000 stETH.
6. On **dock()**, only the recorded amount is released.
7. The extra 3 stETH becomes permanently inaccessible within Aqua.

Scenario 2: Negative Rebase - DoS on Remaining Orders

1. Maker ships 1,000 AMPL into Aqua.
2. Aqua records 1,000 AMPL.
3. A -10% rebase reduces the maker's real balance to 900 AMPL.
4. First taker swaps 500 AMPL successfully.
5. Recorded balance becomes 500 AMPL, real balance 400 AMPL.
6. Second taker attempts to swap 500 AMPL.
7. `safeTransferFrom()` reverts due to insufficient real balance.
8. Remaining order is permanently DoSed.

Remediation:

Consider tracking shares instead of raw balances.

Commentary from the client:

"This is a known model limitation in Aqua balance semantics and handles it as an acknowledged constraint."

OIN15-10 | Extruction Instruction Accepts Arbitrary Target Addresses Without Validation

Acknowledged

Severity:

Informational

Probability:

Rare

Impact:

Informational

Path:

swap-vm/src/instructions/Extruction.sol

Description:

The `_extruction` function reads a target address from maker-signed program data and calls it without validation. A maker can point to any contract, including an upgradeable proxy, so the behavior can change between quote and execution without changing the address. The practical impact is limited by taker-specified `minOut/maxIn` thresholds in `TakerTraits`, but the call itself assumes the target remains trustworthy and stable.

```
function _extruction(Context memory ctx, bytes calldata args) internal {
    address target = address(bytes20(args.slice(0, 20, ExtructionMissingTargetArg.selector)));
    uint256 choppedLength;

    if (ctx.vm.isStaticContext) {
        (ctx.vm.nextPC, choppedLength, ctx.swap) = IStaticExtruction(target).extruction(
            ctx.vm.isStaticContext,
            ctx.vm.nextPC,
            ctx.query,
            ctx.swap,
            args.slice(20),
            ctx.takerArgs()
        );
    } else {
        (ctx.vm.nextPC, choppedLength, ctx.swap) = IExtruction(target).extruction(
            ctx.vm.isStaticContext,
            ctx.vm.nextPC,
            ctx.query,
            ctx.swap,
            args.slice(20),
            ctx.takerArgs()
        );
    }
    bytes calldata chopped = ctx.tryChopTakerArgs(choppedLength);
}
```



```
require(chopped.length == choppedLength, ExtructionChoppedExceededLength(chopped,  
choppedLength)); // Revert if not enough data  
}
```

Remediation:

If stronger guarantees are desired, restrict or bind the target in code, for example by enforcing an allowlist, validating that the target is immutable, or checking a precommitted code hash at execution time. Also document this trust assumption and its implications so integrators and takers understand the expected risk boundary.

Commentary from the client:

"Extruction is documented as a high-flexibility feature with explicit trust/validation responsibilities and "use with care" guidance for integrators.

<https://github.com/1inch/swap-vm/blob/9b9d821eb2f4ed8e2ffa00a4ccb161ca3cef7558/README.md#L557-L563>"

hexens x 1inch"

